

5. Functions

5.1 Simple Functions

5.2 Overloaded Functions and Recursion

5.3 Programs in CodeBlocks

5.1 Simple Functions



1. Built-in & User-defined Functions
2. Passing Arguments to Functions
3. Reference Arguments
4. Returning Values from Functions



- To recognize the use of library functions and user-defined functions
- To know the difference between pass by value and pass by reference

Built-in Functions

➤ Built-in functions are also known as library functions.

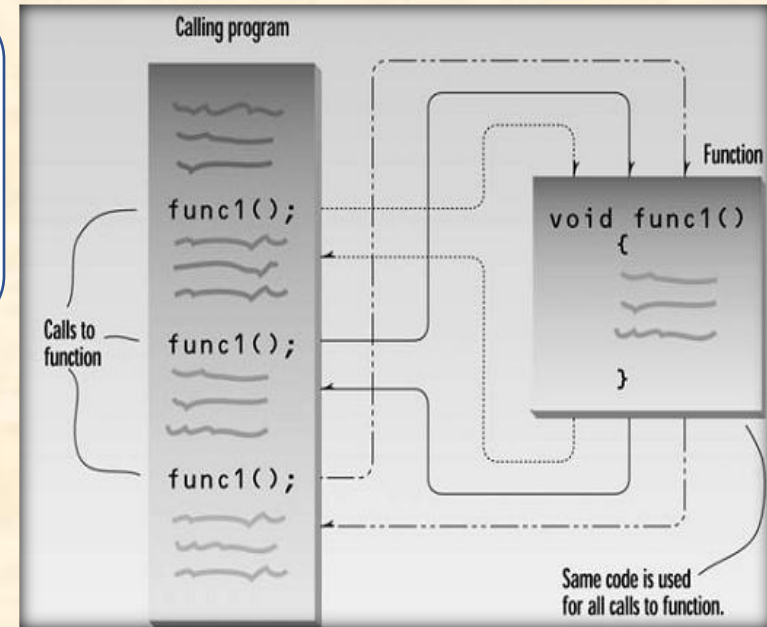
```
#include <iostream>
#include <cmath>
using namespace std;
int main(){
    cout << pow(10,3)<< endl;
    cout << floor(3.65)<< endl;
    cout << ceil(7.25) << endl;
    cout << abs(-105) << endl;
    return 0;
}
```

Output

```
1000
3
8
105
```

User-defined Functions

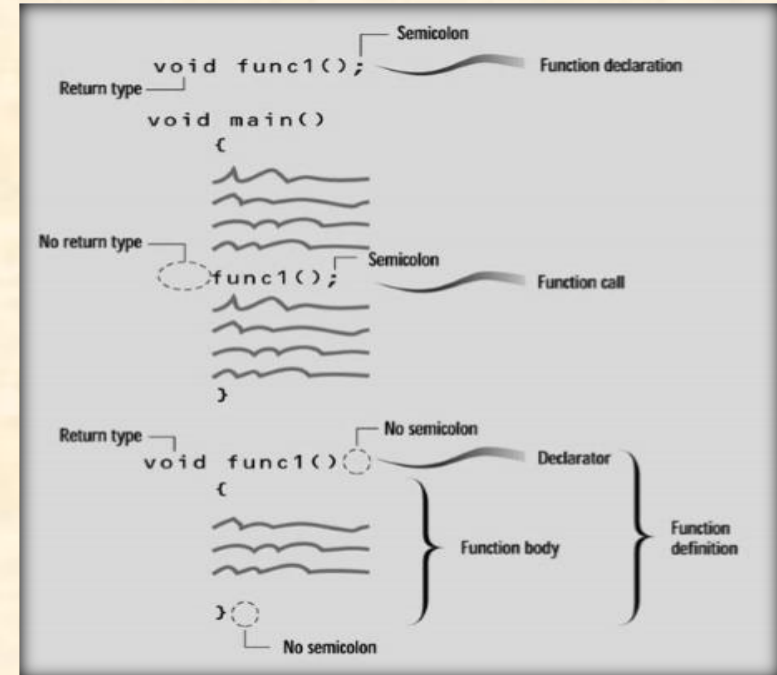
- A user-defined function is a function written by the programmer to perform a specific task.
- It can be called whenever needed in a program.



Function Syntax

```
return_type function_name(parameter_list){  
    // Function body  
    // Statements  
    return value; // Optional for void functions  
}
```

- Function Declaration
- Call to Function
- Function Definition



A Simple Function Example

```
include <iostream>
using namespace std;
void starline();           // function declaration
int main()
{
    starline();             // call to function
    cout << "Data type    Range" << endl;
    starline();             // call to function
    cout <<"char        -128 to 127" << endl
        <<"short      -32,768 to 32,767" << endl
        <<"int        System dependent" << endl
        <<"long       -2,147,483,648 to 2,147,483,647"
        << endl;
    starline();             // call to function
    return 0;
}
```

```
// function definition
void starline()
{
    for(int j=0; j<40; j++)
        cout << '*';
    cout << endl;
}
```

Output

```
*****
Data type    Range
*****
char        -128 to 127
short      -32,768 to 32,767
int        System dependent
long       -2,147,483,648 to 2,147,483,647
*****
```

Eliminating the Declaration

```
include <iostream>
using namespace std;
// function definition
void starline()
{
    for(int j=0; j<40; j++)
        cout << '*';
    cout << endl;
}

int main()
{
    starline(); // call to function
    cout << "Data type Range" << endl;
    starline(); // call to function
}
```

```
cout << "char    -128 to 127" << endl
    << "short   -32,768 to 32,767" << endl
    << "int      System dependent" << endl
    << "long     -2,147,483,648 to 2,147,483,647"
    << endl;

starline(); // call to function
return 0;
}
```

Output

```
*****
Data type   Range
*****
char        -128 to 127
short       -32,768 to 32,767
int         System dependent
long        -2,147,483,648 to 2,147,483,647
*****
```

Passing Arguments to Functions

➤ An **argument** is a piece of data passed from a program to the function.

- Passing by Value- where the function works with a copy of the argument
 - Passing Constants
 - Passing Variables
 - Structures as Arguments
 - Passing Arrays to Functions

Passing Constants

```
#include<iostream>
using namespace std;
void repchar(char, int);    // function declaration
int main()
{
    repchar('=' ,10);      // call to function
    repchar('*' ,15);      // call to function
    repchar('#' ,20);      // call to function
    return 0;
}
void repchar(char ch,int n ) // function definition
{
    for(int j=0;j<n;j++)
        cout<<ch;
    cout<<endl;
}
```

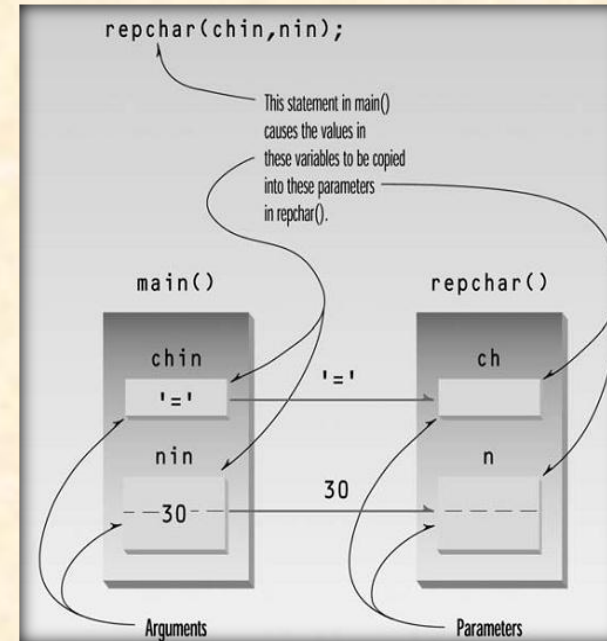
```
=====
*****
#####
```

5.1 Simple Functions

Passing Variables

```
#include<iostream>
using namespace std;
void repchar(char, int);
int main()
{  char chin;    int nin;
   cout<<"Enter character ";
   cin>>chin;
   cout<<"Enter number   ";
   cin>>nin;
   repchar(chin,nin);
   return 0;
}

void repchar(char ch, int n )
{  for(int j=0; j<n; j++)
    cout<<ch;
}
```

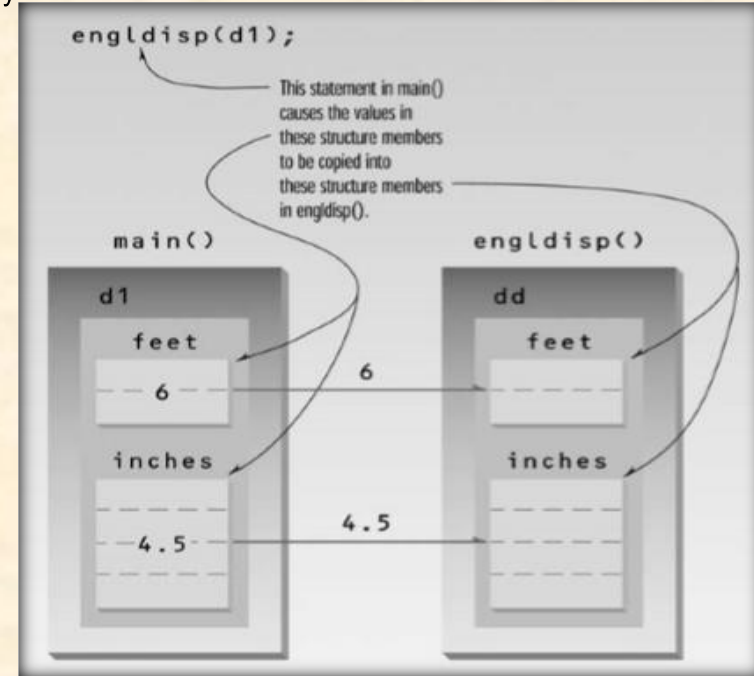


5.1 Simple Functions

Structures as Arguments

d1 = x'y"
d2 = x'y"

```
#include<iostream>
using namespace std;
struct Distance
{   int feet;      float inches;   };
void engldisp(Distance) ;
int main()
{   Distance d1,d2;
    cout<<"Enter feet:";      cin>>d1.feet;
    cout<<"Enter inches:";    cin>>d1.inches;
    cout<<"Enter feet:";      cin>>d2.feet;
    cout<<"Enter inches:";    cin>>d2.inches;
    cout<<"\nd1= ";    engldisp(d1) ;
    cout<<"\nd2= ";    engldisp(d2) ;
    return 0;
}
void engldisp(Distance dd)
{      cout<<dd.feet<<"\' "<<dd.inches<<"\'";  }
```



Passing Arrays to Functions

```
#include <iostream>
using namespace std;
void show(int m[]);
void findAvg(int m[],int size);
int main () {
    int marks[5]={10,12,35,17,56};
    show(marks);
    findAvg(marks,5) ;
    return 0; }
void show(int m[])
{ for (int i = 0; i < 5; ++i)
    cout <<"Student "<<i+1<<": "
    <<m[i]<<endl;
}
```

```
void findAvg(int m[], int size)
{ int i, sum = 0; double avg;
  for (i = 0; i < size; ++i)
      sum += m[i];
  avg = double(sum) / size;
  cout << "Average value is: "
       << avg << endl;
}
```

Output

```
Student 1: 10
Student 2: 12
Student 3: 35
Student 4: 17
Student 5: 56
Average value is: 26
```

Reference Arguments

- **Passing by Reference** - where the function works with the original argument in the calling program.
 - Passing Simple Data Types by Reference
 - Passing Structures by Reference

Passing Simple Data Types by Reference

```
#include<iostream>
using namespace std;
void intfrac(float,float&,float&);
int main()
{
    float number,intpart,fracpart;
    do{
        cout<<"\nEnter a real number: ";
        cin>>number;

        intfrac(number,intpart,fracpart);

        cout<<"Integer part is "<<intpart
            <<" , fraction part is "<<fracpart;
    }while(number!=0.0);
    return 0;
}
```

```
void intfrac(float n,float& intp,float& fracp)
{
    long temp = static_cast<long>(n);
    intp = static_cast<float>(temp);
    fracp = n - intp;
}
```

Program Interaction

```
Enter a real number: 2.5
Integer part is 2, fraction part is 0.5
Enter a real number: 3.78
Integer part is 3, fraction part is 0.78
Enter a real number: 0.0
Integer part is 0, fraction part is 0
```

5.1 Simple Functions

Passing Structures by Reference

```
#include<iostream>
using namespace std;
struct Distance
{   int feet;       float inches; };
void scale(Distance&, float);
void engldisp (Distance);
int main()
{   Distance d1={12,6.5};
    Distance d2={10,5.5};
    cout<<"d1= ";   engldisp(d1);
    cout<<"\nd2= "; engldisp(d2);
    scale(d1,0.5);
    scale(d2,0.25);
    cout<<"\nd1= "; engldisp(d1);
    cout<<"\nd2= "; engldisp(d2);
    return 0;
}
```

```
void scale(Distance& dd , float factor)
{
    float inches=(dd.feet*12+dd.inches)*factor;
    dd.feet=static_cast<int>(inches/12);
    dd.inches=inches-dd.feet*12;
}

void engldisp(Distance dd)
{
    cout<<dd.feet<<"'-"<<dd.inches<<"\"";
}
```

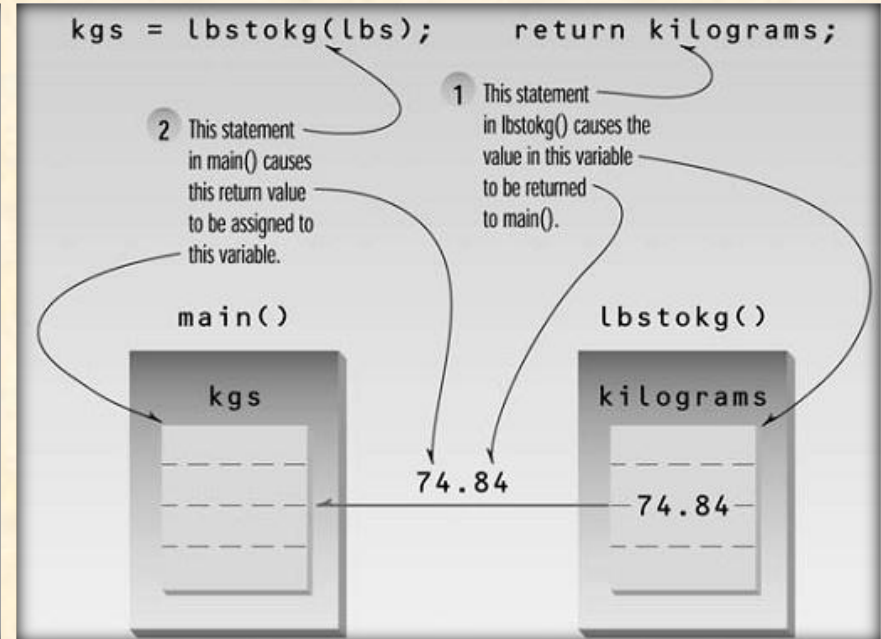
Output

```
d1= 12'-6.5"
d2= 10'-5.5"
d1= 6'-3.25"
d2= 2'-7.375"
```

5.1 Simple Functions

Returning Values from Functions

```
#include<iostream>
using namespace std;
float lbstokg(float);
int main()
{ float lbs,kgs;
  cout<<"Enter your weight in pounds:";
  cin>>lbs;
  kgs=lbstokg(lbs);
  cout<<"Your weight in kilograms is"<<kgs;
  return 0;
}
float lbstokg(float pounds)
{
  float kilograms=0.453592*pounds;
  return kilograms;
}
```



Eliminating Unnecessary Variables

```
#include<iostream>
using namespace std;
float lbstokg(float);
int main()
{   float lbs;
    cout<<"Enter your weight in pounds:";
    cin>>lbs;
    cout<<"Your weight in kilograms is"<<lbstokg(lbs);
    return 0;
}
float lbstokg(float pounds)
{
    return 0.453592*pounds;
}
```

Program Interaction

```
Enter your weight in pounds:182
Your weight in kilograms is82.5537
```

5.1 Simple Functions

Returning Structure Variables

```
#include<iostream>
using namespace std;
struct Distance
{   int feet;        float inches; };
Distance addengl(Distance,Distance) ;
void engldisp(Distance) ;
int main()
{   Distance d1,d2,d3;
    cout<<"Enter feet:";   cin>>d1.feet;
    cout<<"Enter inches:"; cin>>d1.inches;
    cout<<"Enter feet:";   cin>>d2.feet;
    cout<<"Enter inches:"; cin>>d2.inches;
    d3 = addengl(d1,d2) ;      cout<<endl;
    engldisp(d1) ; cout<<"+";
    engldisp(d2) ; cout<<"="; engldisp(d3) ;
    return 0;
}
```

```
Distance addengl(Distance dd1,Distance dd2)
{   Distance dd3;
    dd3.inches=dd1.inches+dd2.inches;
    dd3.feet=dd1.feet+dd2.feet;
    if(dd3.inches >= 12.0)
    {
        dd3.inches -= 12.0;
        dd3.feet++;
    }
    return dd3;
}

void engldisp(Distance dd)
{
    cout<<dd.feet<<"\'- "<<dd.inches<<"\'";
}
```

Quiz 1

If a function returns no value, the return type must be declared as **void**.

A. True

B. False

Answer is A

Quiz 2

Find any errors in the following function definition:

```
int myfunction (int x, y)
{
    return (x*y);
}
```

```
int myfunction (int x, int y)
{
    return (x*y);
}
```

Quiz 3

In case of **pass by value**, it can change the actual parameter value.

- A. True
- B. False

Answer is B

Let's summarize what we learned today

- Functions provide a way to **help organize programs**, and to **reduce program size**, by giving a block of code a name.
- Arguments can be sent to functions either **by value** (a copy of the argument) or **by reference** (original argument in the calling program).
- Functions can return **only one value**.

5. Functions

5.1 Simple Functions

5.2 Overloaded Functions and Recursion

5.3 Programs in CodeBlocks

5.2 Overloaded Functions and Recursion



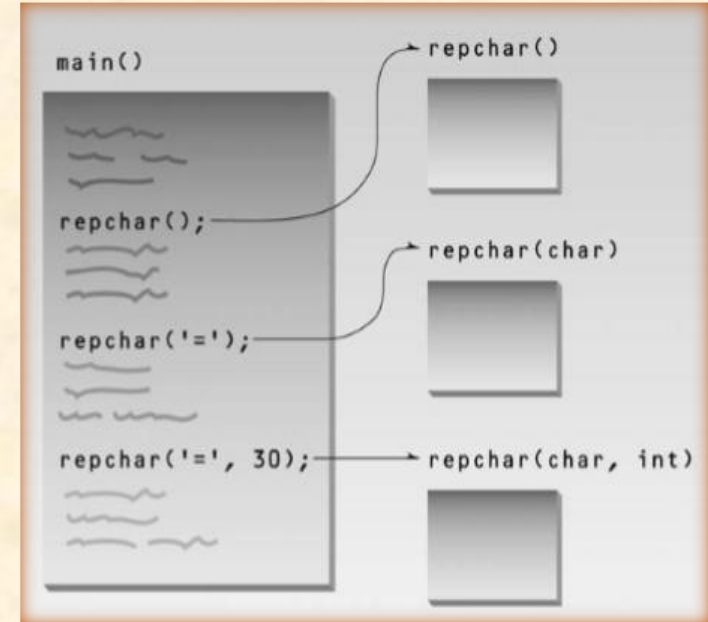
1. Overloaded Functions
2. Recursion
3. Scope and Storage Class



- To know how to overload functions
- To understand how recursion works
- To distinguish the global and local variable usage

Overloaded Functions

- Overloaded Functions means multiple functions can have the same name but different parameter lists (**different number of parameters or different data types**).



5.2 Overloaded Functions and Recursion

Different Numbers of Arguments

```
#include<iostream>
using namespace std;
void repchar();
void repchar(char);
void repchar(char,int);
int main()
{   repchar();
    repchar('=');
    repchar('+',30);
    return 0;
}
void repchar()
{   for(int j=0;j<45;j++)
        cout<<'*';
    cout<<endl;
}
```

```
void repchar(char ch)
{   for(int j=0;j<45;j++)
        cout<<ch;
    cout<<endl;
}
void repchar(char ch,int n)
{   for(int j=0;j<n;j++)
        cout<<ch;
    cout<<endl;
}
```

Output

```
*****
=====
+++++
```

5.2 Overloaded Functions and Recursion

Different Kinds of Arguments

```
#include<iostream>
using namespace std;
void add(int,int);
void add(double,double);
float add(float,float);
int main()
{
    add(3,8);
    add(2.9,5.6);
    float ans=add(1.5F,2.9F);
    cout<<"Float : "<<ans;
    return 0;
}
void add(int a,int b)
{
    cout<<"Int : "<<a+b<<endl;
}
```

```
void add(double x,double y)
{
    cout<<"Double : "<<x+y<<endl;
}

float add(float a,float b)
{
    return a+b;
}
```

Output

```
Int : 11
Double : 8.5
Float : 4.4
```

What is Recursion?

- Recursion is a technique where a function calls itself repeatedly until a base condition is met.

```
void fun1 ()  
{  
    .....  
    fun1 () ;  
    .....  
}  
  
int main ()  
{  
    .....  
    fun1 () ;  
    .....  
}
```

5.2 Overloaded Functions and Recursion

Recursion

```
#include <iostream>
using namespace std;
int fact(int);
int main()
{
    int n,ans;
    cout << "Enter a positive integer: ";
    ;
    cin >> n;
    ans = fact(n);
    cout << "Factorial of "<< n
        << " is " << ans;
    return 0;
}
int fact(int n)
{
    if(n == 0 || n == 1)
        return 1;
    else return n * fact(n-1);
}
```

fact(5) 120

5 * fact(4) 24

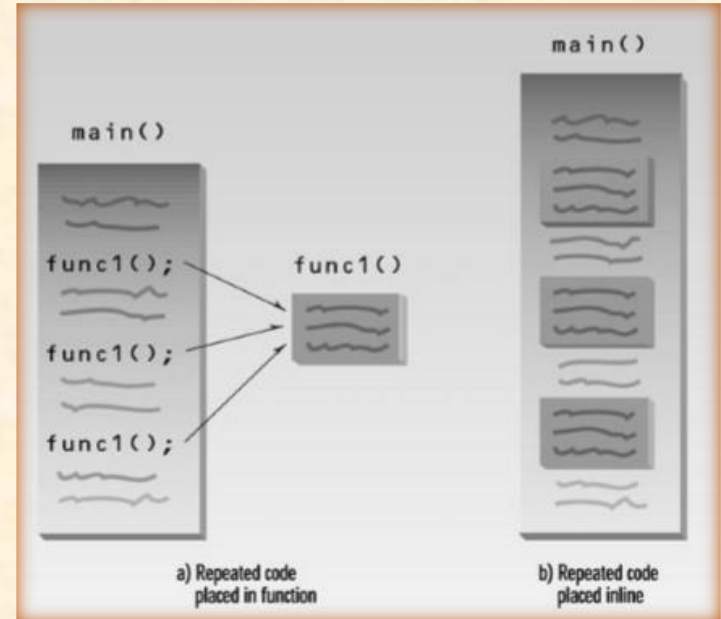
4 * fact(3) 6

3 * fact(2) 2

2 * fact(1) 1

What is Inline Functions ?

- An inline function is a function whose code is substituted at the place where it is called, to reduce function call overhead.



5.2 Overloaded Functions and Recursion

Inline Functions

```
#include <iostream>
using namespace std;
inline float lbstokg(float pounds)
{
    return 0.453592 * pounds;
}
int main()
{
    float lbs;
    cout <<"Enter your weight in pounds: ";
    cin >>lbs;
    cout <<"Your weight in kilograms is "<<lbstokg(lbs);
    return 0;
}
```

Program Interaction

```
Enter your weight in pounds: 132
Your weight in kilograms is 59.8741
```

5.2 Overloaded Functions and Recursion

Default Arguments

- Default arguments are values assigned to function parameters which are used automatically when no argument is passed during function call.

```
#include <iostream>
using namespace std;
void repchar(char '=' , int =45);
int main()
{   repchar();           cout << endl;
    repchar('=');       cout << endl;
    repchar('+', 30);
    return 0;
}
void repchar(char ch, int n)
{   for(int j=0; j<n; j++)
        cout << ch;
}
```

- Default values are given from **right to left**
- void func(int a, int b = 10, int c = 20); // OK
- void func(int a = 5, int b, int c); // Wrong

Output

```
*****
=====
+++++
```

5.2 Overloaded Functions and Recursion

Local Variables

- Local variables are variables declared within a function or block and can only be accessed within that function or block.

```
void somefunc()
{
    int somevar;
    float othervar;
    somevar=10;        //OK
    othervar=11;       //OK
    nextvar=12;        //illegal
}

void otherfunc()
{
    int nextvar;
    somevar=20;        //illegal
    othervar=21;       //illegal
    nextvar=22;        //OK
}
```

5.2 Overloaded Functions and Recursion

Global Variables

- Global Variables are variables that are declared outside all functions and can be accessed by all functions in the program.

```
#include <iostream>
#include <conio.h>
using namespace std;
char ch = 'a';
void getachar();
void putachar();

int main()
{   while( ch != '\r' )
    {   getachar();
        putachar();
    }
    return 0;
}
```

```
void getachar()
{
    ch = getch();
}
void putachar()
{
    cout <<ch;
}
```

5.2 Overloaded Functions and Recursion

Static Local Variables

- Static local variables are local variables declared with the static keyword inside a function, and they keep their values between function calls throughout the execution of the program.

```
#include <iostream>
using namespace std;
float getavg(float);
int main()
{   float data=1, avg;
    while( data != 0 )
    {   cout << "Enter a number: ";
        cin >> data;
        avg = getavg(data);
        cout << "New average is "
            << avg << endl;
    }   return 0;
}
```

```
float getavg(float newdata)
{   static float total = 0;
    static int count = 0;
    count++;
    total += newdata;
    return total / count;
}
```

```
Enter a number: 10
New average is 10      ← total is 10, count is 1
Enter a number: 20
New average is 15      ← total is 30, count is 2
Enter a number: 30
New average is 20      ← total is 60, count is 3
```

Storage Types

	<i>Local</i>	<i>Static Local</i>	<i>Global</i>
Visibility	function	function	file
Lifetime	function	program	program
Initialized value	not initialized	0	0
Storage	stack	heap	heap
Purpose	Variables used by a single function	Same as local, but retains value when function terminates	Variables used by several functions

const Function Arguments

```
#include <iostream>
using namespace std;
void aFunc(int& a, const int& b);
int main()
{   int alpha = 7;
    int beta = 11;
    aFunc(alpha, beta);
    return 0;
}

void aFunc(int& a, const int& b)
{   a = 107;    //OK
    b = 111;    //error: can't modify constant argument
}
```



Quiz 1

Recursion is similar to which of the following?

- A. if...else
- B. if...else if...else
- C. loop
- D. switch case

Answer is C



Quiz 2

A local variable declared in a function is not usable outside that function.

- A. True
- B. False

Answer is A

Quiz 3

Which of the following type of functions is an ideal candidate for being declared **inline**?

- A. A function that is small and is not called frequently.
- B. A function that is small and is called frequently.
- C. A function that is not small and is not called frequently.
- D. A function that is not small and is called frequently.

Answer is B

Let's summarize what we learned today

- An overloaded function is actually a group of functions with the **same name**.
- Recursion - breaking down a problem into **smaller subproblems** in order to make our task easier and feasible
- **Local variables** are visible only within the function.
- **Global variables** are visible throughout an entire file.

Function Practice

1. Write a function named `doubleArray()` that takes an integer array and its size as parameters. The function should double the value of each element in the array.
2. Implement a function named `removeDuplicates()` that takes an integer array and its size (passed by reference) as parameters. The function should remove duplicate elements from the array and update the size accordingly.
3. Write a recursive function called `countDigits()` that takes an integer parameter and returns the count of digits in the number. The function should use recursion to determine the number of digits.
4. Implement a function called `findMax()` that takes an array of elements and its size as parameters. The function should return the maximum element in the array.
5. Write overloaded functions named `larger()` that take two parameters of the same type and return the larger value. Provide overloaded versions for the following data types: `int`, `double`, and `char`.
6. Write overloaded functions named `area()` to calculate the area of different geometric shapes. Provide overloaded versions with the following parameter lists:
 - `area(double length, double width)` $\leftarrow$ Rectangle Area = $\text{length} \times \text{width}$
 - `area(double radius)` $\leftarrow$ Circle Area = $\pi \times \text{radius}^2$
 - `area(double base, double height, char shape)` $\leftarrow$ Triangle Area = $(\text{base} \times \text{height}) / 2$